ID	Notes & Description	Example/s
1	When the document is returned (not delivered), it should be registered in WMS, Finance, and Return Request.	
2	Review all chats and calls that are related to any task or bug.	Collect the emails, messages, voice chats, comments, and tasks for all blocked issues. **Ex-01**: The issue of matching the WMS, POD, Employees Portal, and the Finance Applications. **Ex-02**: Test all options for the select lists, including the product and category preview issue. **Ex-03**: Test all options for the warehouse types. **Ex-04**: Test all the options for creating a consignment from the Sales application one by one or by groups. **Ex-05**: Test all the task types.
3	When there is any change to data that is related across multiple screens, check: 1. The details screen on all pages. 2. The related and parent screens.	**Ex-01**: The wallet payout between web and mobile. **Ex-02**: The return request reason field between the details API and the Edit API. **Ex-03**: The consignment status between web and mobile when sending extra information from mobile to web. **Ex-04**: The portal interface between web, external domain, and mobile. **Ex-05**: When there is a difference between screens, always report the issue to the board.

		Ex-06: When the object has multiple statuses, check its operation for create, edit, change status, change quantity, change amount, etc. **Ex-07**: When we have draft objects like order, expense, invoice, etc., check the details after approval, like the Goods Receipt, the payment amount, status value, etc. **Ex-08**: When we have general functionalities like add payment, approve order, approve consignment, etc., match the feature to the object's status.
4	The effect of **Edit** on **the related data**, such as **total**, **subtotal**, **children**, **accounts**, **trial balance**, **status**, etc.	**Ex-01**: The header discount and the line discount. **Ex-02**: Draft report with an existing payment; then we need to approve it. **Ex-03**: The optional fields and their effects on the other fields. • Like we want to update the coupon field then remove it. • The country field in the Address component and its relation to the other fields. **Ex-04**: The report and its expenses; we don't need to refresh the page to change the status of the report. **Ex-05**: When we have multiple steps, ways, etc., to update the account information in 1-way or more, check the list and details screen or dialogues. **Ex-06**: Check the validations for the key labels.

5	When testing the dashboard pages, we should check: 1. The charts. 2. The statistics data. 3. The summary data. 4. The total values. 5. The report shapes. 6. The refreshed data on mobile after creating new data.	**Ex-01**: The score of the candidate in the HR portal screen when we have multiple factors that affect it. **Ex-02**: The application of the candidate when importing the data. **Ex-03**: The labels of the data when handling the UI/UX issues. **Ex-04**: Check the multi-select list if they have checkboxes or not, they may not need them at all.
6	When we have roles and permissions with/without related sector/s like the branches of the company, check the related menu roles and permissions.	**Ex-01:** The normal customers, VIP customers. **Ex-02**: The pending expenses, the draft expenses, the approved expenses. **Ex-03**: The related objects that share the same API with different content, like the Documents and the checkout API.
7	When we have related data, test: 1. The export column name in the file. 2. The export column data in the file. 3. The metafield columns for displaying, exporting, creating, and editing. 4. The delete option, if it exists. 5. Check whether we delete the related data if we can regenerate it in the same process. 6. Check whether we update the related data if we can regenerate it in the same process. 7. The email template with the profile data and images, such as the company name, the company logo, the company website, the company VAT, etc. 8. Check the currency of the total, subtotal, warranty plans, tax values, etc.	

8	1. Check the export object as a PDF file. 2. Check the exported data against the file data to confirm they are the same and correct.	**Ex-01:** Check the file title. **Ex-02:** Check the file details. **Ex-03:** Check the file template, images, communication fields, etc.
9	Check the Cache issue in the UI when the user cancels the option: whether the form is cleared or not. Check The Cache issue from the BE side. Check the API response times from the UI and Postman.	
10	Check the background jobs and schedulers that may send invalid data instead of the correct one, like email data, reminders, email file/s, etc.	
11	Test repeating the case multiple times to check the result on the details screen or via the API.	**Ex-01**: The termination request and cancel one. **Ex-02**: The duplication of documents with the online payment gateways. **Ex-03**: The related metafield to the selected type, such as the bin-managed warehouses and their structure and tree.
12	When creating or updating the relation data, check: 1. The self-relation with the item itself. 2. The reflective relation between child and parent, and parent and child.	
13	When testing Finance and Account Applications, check the details of each account and the Debit/Credit amounts.	**Ex-01:** Like Accounts Receivable and Accounts Payable.
14	When testing the WMS applications, we should check: 1. The finance events. 2. The Inbound/Outbound operations. 3. The affected warehouses. 4. The effect of serial/batch products.	**Ex-01**: The effect of big numbers in the fields. **Ex-02**: The effect of empty data in the fields. **Ex-03**: Faulty documents with zero amounts.

	5. The effect of batch and serial products. 6. The effect of BOM products. 7. Withdrawing goods from the vendor or customer. 8. The effect of approved return requests on Inbound/Outbound operations and the warehouse's availability. 9. The effect of Order/Delivery documents.	**Ex-04**: Faulty products with no warehouse availability at all. **Ex-05**: The effect of draft data, like in consignments and orders, and how we can add them.
15	When testing any **Financial Report Services**, check the balance in these situations: 1. The total Fiscal Year. 2. Monthly Period. 3. Yearly Period. 4. Custom Period. Like the account statement report.	**Ex-01**: Try to generate random data for each fiscal year, and check the effects on the account statement report. **Ex-02**: The same as #01 but for all other types of financial reports.
16	For sorting and rearranging **items**, **products**, **pages**, etc. 1. Check the different types and the effect of each of them. 2. Check the different sections of the page, items, etc.	**Ex-01**: Try to change the item in the same section. **Ex-02**: Try to change the item to another section, and check the effect on the details and edit screens.
17	When we have validation issues, check: 1. The validation from the FE matches the validation from the BE side. 2. The validation on the mobile matches the validation on the web and the BE. 3. The validation rules when all the fields are empty. 4. The validation rules for related ones, like the coupon, and changing it multiple times.	**Ex-01:** The address component: when adding a country, we must check the related validation rules for the other fields.
18	The user wants to open all available screens in the same window.	
19	Check the color of the data status.	**Ex-01:** The rating of the candidate's score. **Ex-02:** The matching score of the candidate's skills.
20	When we have an API for the details of the object, test all the scenarios of: 1. Create A New One From/To it. 2. Duplicate the current one. 3. Edit The Selected One.	

	4. Change its status. 5. Display the result on the web & mobile if it exists. 6. Check the related data that was sent from the external source, like mobile, and the data that was sent. 7. Check the online payment gateways if they are available.	
21	Collect the page URLs, buttons, and other links to other pages and redirects.	**Ex-01**: The Payout page: the Expenses list screen, wallet details screen, history log of expense details screen, in the table of the expense details screen, and the nav menu. **Ex-02**: The consignment page: in the POD web application, in the Sales web application, and in the POD mobile application. **Ex-03**: The documents: the POD application and the Employees Portal application. **Ex-04**: Collect the status of each object, such as: consignments, Goods Receipt, Goods Issue, Expense Payment, Reimbursed Payment, etc. **Ex-05**: Check the hyperlinks in the summary and collection tables, such as Wallet details, transaction histories, etc.
22	Check the status of the object.	**Ex-01:** The Order document: the POD application, the Employees Portal application, and the related Business Entity details page. **Ex-02:** Check the chat for the predefined statuses of objects like Sales. **Ex-03:** The configurable products and their related warranty templates.

		Ex-04: The status of the task between the tasks screen in the mobile (**pending**) and the route screen (**pending** & **in-progress**)
23	For the portals and custom domains, we should check the status.	**Ex-01:** The application should check the status on creation. **Ex-02:** The application should display the status information. **Ex-03:** The application should allow the user to re-check the domain status.
24	Check the number of requests that are sent from the **application**, **module**, **entity type**, **sort**, **filter**, **create screen**, **edit screen**, etc.	**Ex-01:** When we create an object, does the page send any unused requests? **Ex-02:** When we display the details of the object, does the page send any unused requests? **Ex-03:** The Save and Save as Draft buttons stayed disabled until that request finished in create, edit, change-status, etc. **Ex-04**: Do the edit, change status, add new permission, and create pages fill the forms with default values? **Ex-05**: Check the loader or skeleton on the fields of create, edit, change status, etc. for the fields that require API calls. **Ex-06**: Check for duplicate API requests. • When the user opens a page, an API is called twice; one in the begin of **loading page**, and

		the second when **focus on the field to search**.
25	When the entity type has attachment object/s, check the display of this attachment in the list, details, assets, and edit screens.	**Ex-01:** The user displays the attachment/s on the list screen; should we display them with budget or not? **Ex-02**: The user displays the details screen; how should we display the attachment/s in a carousel or a separate component?
26	When testing the e-commerce documents, or related entity objects that have special parameters, check the create, details, and edit screens, then the details screen.	**Ex-01:** Check the validation on create and edit screens for special params, such as the Purchase Number. **Ex-02**: Check the details screen after creating or editing the entity object. **Ex-03**: When there are metafield values, check the details screen after performing create, edit, change status, duplicate, etc.
27	When we have labels for any status column, check whether the labels of the status columns are unified or not.	**Ex-01**: The payment status of the expense. **Ex-02**: The reimbursement status of the expense. **Ex-03**: The document payment status.
28	Check the API response for each request that changes the entity object during create, edit, change status, change permission, and compare it with the optimal scenario if the FE requests any API related to getting the contents by ID or any other way.	**Ex-01:** If we have a cart for the e-commerce documents, check the response for the cart API endpoints. **Ex-02**: If the expense amount was changed, check all the requested APIs.
29	When the user duplicates the page in the browser, ensure that the related data are synced between FE and BE	**Ex-01:** When the user opens a cart and duplicates the page, check the cart items, cart status, payment amount, etc., after checkout.

30	Check the format of dates, numbers, labels, amounts, etc., in the application.	
31	In the multi-currency system, check the currency object with the ISO code.	**Ex-01:** The document data. **Ex-02:** The paid, top-up, and reimbursement amounts of the wallets.
32	When we have multiple ways to create content, we need to check the details, edit, change status, etc., forms for each way.	**Ex-01:** The user can create or generate a skill using the AI, so we need to check the details and edit forms.
33	When the user creates a document and pays, check the Journal Entry or the Financial effect of the payment.	
34	When the user uploads an attachment, image, document, etc., check the functionality of viewing, downloading, deleting, etc. for the uploaded one.	
35	When the user cancels a document, examine the financial entry corresponding to the document in question.	
36	When the user uses AI automation to scan documents, check the prompt and the final result. Match the result with the current fields, and ask if we can add the missing fields or create them from the BE side.	
37	1. When we have a payment module that is related to the E-Commerce Documents module, we should check the paid amount during reconciliation. 2. We should check the Open Item or Remaining Open Amount. 3. We should check the validation and permission rules for each employee to see if he/she can make the reconciliation with the remaining amount or not (refer to the chat file for more information)	

38	When handling e-commerce documents, check all the ways to create, edit, delete, etc., that make the application work.	**Ex-01:** If we want to remove a feature from the customer application, we can follow the main flow to check the full scenarios.
39	Check the optional and auto-complete fields after resetting them in create, edit, change status, change permission/s, etc.	**Ex-01:** Check the empty string fields against the null values, such as the price list of the business entities.
40	When handling Performance Testing & Automation Testing of gaming applications, we should: 1. Simulate the player strategy. 2. Simulate the bot strategy. 3. Check all the available APIs of the game SDK. 4. Simulate slower networks. 5. Check the cached data & graphics before and after testing. 6. Test the statistical data of the game with screenshots.	Search for the state of testing mobile games for these situations: 1. Manual Testing. 2. Automation Testing. 3. Performance Testing. 4. Simulate bots. 5. Building gaming bots. 6. Using game SDK. 7. All other related fields.
41	When creating a communication channel, check for duplication of the email and type, JSON configuration file & type, etc.	**Ex-01**: The organization email address and the communication channel type.
42	After we finish testing, we should begin with: Exploratory Testing	**Ex-01:** Use the chat bots, the chat agents, search web features of the chat bots, search engines, etc., to handle the future requirement, and observe the work of the tool.
43	These tests are designed to test the non-functional aspects of a system and include: • Security Testing. • Performance Testing. • Compatibility Testing. • Compliance Testing. • Usability Testing.	

44	Types of functional tests primarily include: • Acceptance Testing. • System Testing. • Integration Testing. • Interface Testing. • Smoke Testing. • Unit Testing. • And others.	
45	When testing e-commerce documents check the validation of fields for each type.	**Ex-01:** The quotation should have no validation at all. **Ex-02**: The order should have configurations for signature, and all the fields are required. **Ex-03**: The order document should be trucked if we have an active payment on it. **Ex-04**: Check the warehouse distribution for the delivery document. **Ex-05**: Check the Goods Issue for the approved return request. **Ex-06**: Check the credit limit for the selected customer for credit note.